Li Shuhao

: shuhao.li@inf.ethz.ch
: github.com/Yanksi
: +41 (078)255-85-55

Education

ETH Zurich — Zurich, Switzerland
Ph.D in Computer Science — 2025–now
Supervisor: Prof. Torsten Hoefler

ETH Zurich — Zurich, Switzerland
M.S. in Computer Science, GPA: 5.60/6.00 — 2022–2025

University of Toronto — Toronto, Ontario, Canada
B.S. in Computer Science, average GPA: 3.88/4.0 — 2016–2021

Graduated with a Honored Degree with High Distinction.
Focusing on Artificial Intelligence, Computer Vision, and Scientific Computing.

Research Experience

PrismLinear — Zurich, Switzerland
Ongoing PhD Research Project, Scalable Parallel Computing Laboratory, ETH Zurich — 2026–now

- Proposed a block-diagonal grouped linear layer that, when paired with a gated SiLU activation, serves as a parameter-efficient width-expansion primitive for network pretraining: each layer provides multiple independent pathways at a fraction of the parameter and compute cost of an equivalently-wide dense MLP.
- Implemented a fused batched-GEMM + GEMM forward kernel in CuTe for the proposed layer, computing $(AR^\top)B^\top$ with block-diagonal R in a single kernel launch. On RTX 3080 Ti, the forward kernel matches cuBLAS GEMM runtime at the equivalent dense (M, N, K) problem size and achieves $>3\times$ speedup over an optimized cuBLAS baseline that requires two cuBLAS calls per group ($2n_{\text{groups}}$ kernel launches in total).
- Used Nsight Compute to profile warp-level execution and identified distinct throughput bottlenecks before each computation stage: memory-bound stalls preceding the batched-GEMM (AR) and compute-bound stalls (tensor-core contention) preceding the subsequent GEMM. Restructured the kernel into a producer–consumer warp model—memory-heavy and compute-heavy work assigned to separate warp groups with PTX-level barrier coordination (`bar.sync`, `bar.arrive`)—to overlap the two resource profiles and improve overall occupancy.
- Implemented matching fused backward kernels for the A/R and B gradients, integrated with PyTorch via a custom autograd op. Built an end-to-end autotuning system with pipelined JIT compilation and benchmarking (producer–consumer overlap, multi-GPU support, on-disk result cache) that selects optimal tile sizes, pipeline depths, and warp layouts per problem shape.

NCCL Trace Generation for Distributed Training Simulation — Zurich, Switzerland
PhD Research Project, Scalable Parallel Computing Laboratory, ETH Zurich — 2025–2026

- Re-architected a translator from Nsight Systems traces to the Goal trace format used by the LogGOPSim network simulator. Replaced a monolithic 2000-line procedural script with a domain-driven object model (physical, NCCL, and Goal IR layers), enabling extensibility to alternative collective implementations and per-communication context labeling.
- Resolved critical performance regressions introduced by the richer object model: applied fork-based multiprocessing with copy-on-write–friendly immutable data layouts to achieve parallelism without serialization overhead, and lazy evaluation via Python generators to bound memory consumption on large-scale traces.
- Developed a lightweight training-simulation framework (`simple_sim`) with a PyTorch-like API and a built-in autograd engine that supports communication operators (AllReduce, ReduceScatter, AllGather), enabling synthetic trace generation with roofline-modeled compute costs for workloads beyond available hardware scale.

Replacing Dense Layers with Higher-Efficiency Structures Zurich, Switzerland
Master Thesis, Scalable Parallel Computing Laboratory, ETH Zurich 2024–2025

- Investigated structured-sparsity-aware linear-layer parameterizations that exploit NVIDIA Sparse Tensor Cores (2:4 structured sparsity) for higher training and inference throughput on Ampere GPUs.
- Generalized orthogonal fine-tuning (OFT) to block-structured variants that admit substantially higher kernel efficiency while preserving the orthogonal-constraint benefits of the original formulation.
- Empirically showed that block-diagonal layer structures can be used during pretraining to recover the accuracy gap of 2:4-sparse models, bridging pretraining-time architecture design with post-training hardware sparsification.

Skills

- **Programming:** C/C++, CUDA, PTX, Python, OpenMP, MPI
- **GPU & HPC:** CuTe, CUTLASS, tensor-core programming (Ampere MMA), GPU kernel autotuning, profiling-driven optimization (Nsight Compute), warp-level synchronization primitives
- **ML & Frameworks:** PyTorch (custom autograd ops, C++/CUDA extensions), Deep Learning

Work Experience

Noah's Ark Laboratory, Huawei Technologies Canada Markham, ON, Canada
Research Engineer (Contract) 2021–2022

- Contributed to the development of an on-device eye-tracking algorithm for smartphone applications. Implemented the model-calibration procedure using Newton's method for fast per-user adaptation and designed the accompanying gaze-following calibration protocol with verified fixation tracking.
- Built an end-to-end Android application that captures calibration data via the designed protocol, runs on-device calibration, and demonstrates live gaze-tracking performance with the calibrated model.

Noah's Ark Laboratory, Huawei Technologies Canada Markham, ON, Canada
Research Intern 2019–2020

- Led the development of a smartphone-based paint color estimation method, resulting in a granted US patent (US11810329B2) with the author as a listed inventor.
- Proposed a flash-based measurement protocol that recovers the environment-invariant spectral response of a paint sample by pairing a flash-lit capture with an ambient capture.
- Developed Android demo applications to showcase the team's computer-vision algorithms.

Projects

Polybench Kernel Optimization Zurich, Switzerland
Course Project, Design of Parallel and High-Performance Computing, ETH Zurich 2024

- Implemented a CUDA tensor-core kernel for symmetric matrix–matrix multiplication (`symm`) using the WMMA API with TF32 precision. Exploited the symmetry of A at the tile-loading stage—below-diagonal, above-diagonal, and diagonal-crossing tiles each follow a distinct shared-memory layout path—to halve redundant global-memory traffic while preserving vectorized 128-bit loads and correct WMMA fragment alignment.
- Achieved 2.1× speedup over cuBLAS `symm` on RTX 4090 by combining symmetry-aware tiling with multi-level blocking (thread-block, warp, and WMMA-fragment granularity).
- Co-developed a fused CPU kernel for `gemver` that consolidates the four constituent BLAS calls into a single pass, achieving 8× speedup over composed OpenBLAS kernels on AMD EPYC 7H12 by saturating memory bandwidth with minimal redundant traffic.